

---

**IPCA**  
**Instituto Politécnico do Cávado e do Ave**  
**Ano Letivo 2025/26**

---

Curso Sistemas Eletrónicos e Computadores

# Multímetro com Datalogger

Laboratório e Desenvolvimento de Hardware

Guilherme Marques

Dinis Ribeiro

Gonçalo Barbosa

**Prof. Orientador: Carlos Ferreira**

**Carlos Silva**

## Resumo

Este relatório descreve o desenvolvimento de um multímetro com datalogger, realizado no âmbito da unidade curricular de *Laboratórios de Desenvolvimento de Hardware*. O sistema desenvolvido é capaz de medir tensão no intervalo de 0 a 30 V e corrente contínua entre 0 e 3 A, apresentando os valores através de três displays de 7 segmentos multiplexados e enviar os dados para um computador por comunicação série.

O projeto envolveu todas as fases de desenvolvimento de um sistema eletrónico embebido, incluindo a definição dos requisitos, conceção do hardware, dimensionamento de componentes, desenho dos esquemáticos e da PCB, desenvolvimento do software e realização de testes. O microcontrolador ATmega328P é responsável pela aquisição dos sinais analógicos, cálculo das médias configuráveis e controlo da interface de visualização e comunicação.

Apesar de algumas limitações encontradas, nomeadamente na implementação do baud rate pretendido de 115200 bps, o sistema final cumpre os principais requisitos definidos, permitindo uma medição estável e configurável das grandezas elétricas pretendidas.

## Índice

|                                                            |    |
|------------------------------------------------------------|----|
| Resumo.....                                                | 2  |
| Índice de Figuras .....                                    | 5  |
| Notação e glossário.....                                   | 6  |
| 1. Introdução.....                                         | 7  |
| 2. Objetivos .....                                         | 8  |
| 3. Âmbito do Projeto.....                                  | 9  |
| 4. Metodologia de desenvolvimento .....                    | 10 |
| 5. Requisitos do Sistema .....                             | 11 |
| 5.1. Medição .....                                         | 11 |
| 5.2. Apresentação.....                                     | 11 |
| 5.3. Configuração e Comunicação .....                      | 11 |
| 6. Arquitetura do sistema .....                            | 12 |
| 6.1. Diagrama de blocos.....                               | 12 |
| 7. Hardware.....                                           | 13 |
| 7.1. Pesquisa dos footprints .....                         | 13 |
| 7.1.2. 2ºPasso: Importação das bibliotecas.....            | 13 |
| 7.1.3. 3ºPasso – Ajusto dos PADs.....                      | 14 |
| 7.2. Organização da PCB .....                              | 14 |
| 7.3. PCB.....                                              | 15 |
| 7.4. ATmega328P .....                                      | 16 |
| 7.5. FTDI.....                                             | 17 |
| 7.6. Sensor de corrente .....                              | 18 |
| 7.7. Divisor de tensão.....                                | 19 |
| 7.8. Display's.....                                        | 20 |
| 7.9. LED's e Botões .....                                  | 21 |
| 7.10. Esquema final.....                                   | 22 |
| 8. Software .....                                          | 23 |
| 8.1. Definição das bibliotecas.....                        | 23 |
| 8.2. Variáveis globais.....                                | 23 |
| 8.3. Padrão de bits dos segmentos dos display's.....       | 23 |
| 8.4. Atualização dos display's (interrupção do timer)..... | 24 |
| 8.5. Inicialização serial.....                             | 24 |
| 8.6. Função para enviar texto para o PC .....              | 25 |
| 8.7. Função para processar comandos recebidos .....        | 25 |

|       |                             |    |
|-------|-----------------------------|----|
| 8.8.  | Função leitura do ADC ..... | 26 |
| 8.9.  | Função principal .....      | 26 |
| 8.10. | Loop .....                  | 27 |
| 9.    | Testes.....                 | 30 |
| 9.1.  | Corrente .....              | 30 |
| 9.2.  | Tensão.....                 | 31 |
|       | .....                       | 31 |
| 10.   | PCB Final .....             | 32 |
| 11.   | Conclusão.....              | 33 |

## Índice de Figuras

|                                                            |    |
|------------------------------------------------------------|----|
| Figura 1 - Diagrama de Blocos .....                        | 12 |
| Figura 2 - Pesquisa dos componentes .....                  | 13 |
| Figura 3 - Footprint do display .....                      | 13 |
| Figura 4 - Schematic Display .....                         | 13 |
| Figura 5 - Pads do ATmega328P .....                        | 14 |
| Figura 6 - Rooms do Altium .....                           | 14 |
| Figura 7 - PCB sem componentes .....                       | 15 |
| Figura 8 - ATmega328P .....                                | 16 |
| Figura 9 - Schematic ATmega328P .....                      | 16 |
| Figura 10 - FT232RL .....                                  | 17 |
| Figura 11 - Schematic do FTDI .....                        | 17 |
| Figura 12 - ACS712 .....                                   | 18 |
| Figura 13 - Schematic Sensor de corrente .....             | 18 |
| Figura 14 - Schematic divisor de tensão .....              | 19 |
| Figura 15 - Schematic dos display's .....                  | 20 |
| Figura 16 - LED's e botões .....                           | 21 |
| Figura 17 - Esquema final.....                             | 22 |
| Figura 18 - Definição das bibliotecas .....                | 23 |
| Figura 19 - Variáveis globais .....                        | 23 |
| Figura 20 - Padrão de bits dos segmentos.....              | 23 |
| Figura 21 - Atualização dos display's.....                 | 24 |
| Figura 22 - Inicialização serial.....                      | 24 |
| Figura 23 - Função para enviar texto para o PC.....        | 25 |
| Figura 24 - Função para processar comandos recebidos ..... | 25 |
| Figura 25 - Leitura do ADC.....                            | 26 |
| Figura 26 - Função principal .....                         | 26 |
| Figura 27 - Leitura do botão .....                         | 27 |
| Figura 28 - Atualização de dados nos display's .....       | 28 |
| Figura 29 - Calculo da média para o serial monitor.....    | 29 |
| Figura 30 - Teste da media Monitor Serial .....            | 30 |
| Figura 31 - Teste funcional de corrente .....              | 30 |
| Figura 32 - Teste na fonte de alimentação.....             | 31 |
| Figura 33 - Teste com a pilha .....                        | 31 |
| Figura 34 - Teste do comando no monitor serial .....       | 31 |
| Figura 35 - PCB Final.....                                 | 32 |

## Notação e glossário

IPCA- Instituto Politécnico do Cávado e do Ave

# 1.Introdução

O presente relatório foi elaborado no âmbito da unidade curricular Laboratórios de Desenvolvimento de Hardware e tem como objetivo documentar todo o trabalho desenvolvido no Multímetro com Datalogger. Este documento reúne a descrição do problema, o planeamento do projeto, a análise e o design do sistema, bem como as etapas de fabrico, teste e avaliação, seguindo a metodologia definida pelos docentes.

O projeto consiste no desenvolvimento de um sistema eletrónico embebido capaz de medir tensão e corrente, apresentar os valores localmente e enviar os dados através de comunicação série, respeitando um conjunto de requisitos técnicos previamente definidos

## 2. Objetivos

O principal objetivo deste projeto é o desenvolvimento de um multímetro digital com datalogger para medição de tensão e corrente contínuas. De forma mais específica, pretende-se:

- Medir tensão entre 0 e 30 V com resolução de 0.1 V;
- Medir corrente entre 0 e 3 A com resolução de 0.1 A;
- Mostrar os valores medidos em displays de 7 segmentos;
- Utilizar LEDs para indicar o tipo de medição (tensão ou corrente);
- Permitir configurar o número de amostras usadas para o cálculo da média de tensão e corrente;
- Permitir configurar a frequência de envio dos dados pela porta série;
- Implementar comunicação serial com baud rate de 115200 bps;

### **3.Âmbito do Projeto**

O projeto foi desenvolvido sob a forma de um protótipo funcional, englobando a medição de tensão e corrente, a visualização dos valores em três displays de 7 segmentos multiplexados e a indicação do modo de medição através de LEDs. O trabalho incluiu o desenvolvimento de hardware, nomeadamente os circuitos de medição e aquisição de dados, bem como o desenvolvimento de software para processamento das medições e comunicação com um computador.

Estão incluídos no âmbito do projeto:

- Circuitos de medição de tensão e corrente;
- Conversão analógico-digital;
- Interface de visualização com displays de 7 segmentos;
- Comunicação série com um computador;
- Firmware desenvolvido em linguagem C.

## **4. Metodologia de desenvolvimento**

O desenvolvimento do projeto seguiu uma metodologia estruturada por fases, adequada à implementação de sistemas eletrónicos embebidos. Numa fase inicial, foi realizada a análise do problema e a definição dos requisitos funcionais e técnicos do sistema. De seguida, procedeu-se à conceção da arquitetura global e ao dimensionamento dos diferentes blocos de hardware.

Numa fase posterior, foram desenvolvidos os esquemáticos elétricos e o layout da placa de circuito impresso (PCB), recorrendo ao software Altium Designer. Após a fabricação da PCB, procedeu-se à soldadura dos componentes. Em paralelo, foi desenvolvido o firmware em linguagem C, permitindo uma validação progressiva do sistema. Por fim, foram realizados testes funcionais com o objetivo de verificar o correto funcionamento do protótipo e avaliar o seu desempenho face aos requisitos definidos

## **5. Requisitos do Sistema**

O sistema desenvolvido deverá cumprir os seguintes requisitos:

### **5.1. Medição**

- Medição de tensão no intervalo de 0 a 30 V, com resolução de 0,1 V;
- Medição de corrente no intervalo de 0 a 3 A, com resolução de 0,1 A;

### **5.2. Apresentação**

- Apresentação dos valores medidos em três displays de 7 segmentos multiplexados;
- Utilização de dois LEDs para indicação da grandeza em medição (tensão ou corrente);

### **5.3. Configuração e Comunicação**

- Receção de comandos pela porta série para:
  - Configurar o número de amostras usadas na média da medição de corrente;
  - Configurar o número de amostras usadas na média da medição de tensão;
  - Configurar a frequência de envio dos dados;
- Comunicação série com baud rate de 115200 bps;

## 6.Arquitetura do sistema

### 6.1. Diagrama de blocos

A Figura 1 apresenta o diagrama de blocos do multímetro com datalogger desenvolvido. O sistema é constituído por vários blocos funcionais responsáveis pela alimentação, aquisição das grandezas elétricas, processamento das medições, apresentação local dos resultados e comunicação com um computador.

- O bloco de Alimentação fornece a energia necessária ao funcionamento do sistema. O Controlador, baseado no microcontrolador ATmega328P, constitui o núcleo do sistema, sendo responsável pela aquisição dos sinais analógicos, conversão analógico-digital, processamento das medições e controlo das interfaces de visualização e comunicação;
- A medição de tensão e corrente é realizada através de circuitos de condicionamento de sinal ligados ao Sistema a ser Medido, permitindo adaptar as grandezas elétricas aos níveis suportados pelo ADC do microcontrolador. A medição de tensão é efetuada através de um divisor de tensão, enquanto a medição de corrente é realizada através do sensor de corrente ACS712, baseado no efeito Hall;
- Os valores medidos são apresentados nos Displays que são constituído por três displays de 7 segmentos multiplexados. Adicionalmente, o sistema comunica com um Computador através de uma interface série, permitindo o envio das medições e a receção de comandos de configuração;



Figura 1 - Diagrama de Blocos

## 7. Hardware

### 7.1. Pesquisa dos footprints

Antes do desenho final da placa de circuito impresso, foi realizada a pesquisa e validação dos footprints de todos os componentes utilizados, garantindo a compatibilidade com os encapsulamentos físicos reais e evitando erros na fase de montagem. Esta etapa revelou-se fundamental para assegurar a correta implementação do layout da PCB.

#### 7.1.1. 1ºPasso – Pesquisa dos componentes

Inicialmente, foi efetuada a pesquisa dos componentes necessários ao projeto, recorrendo aos fabricantes e a bibliotecas online. Após a identificação correta de cada componente, procedeu-se ao download das respetivas bibliotecas

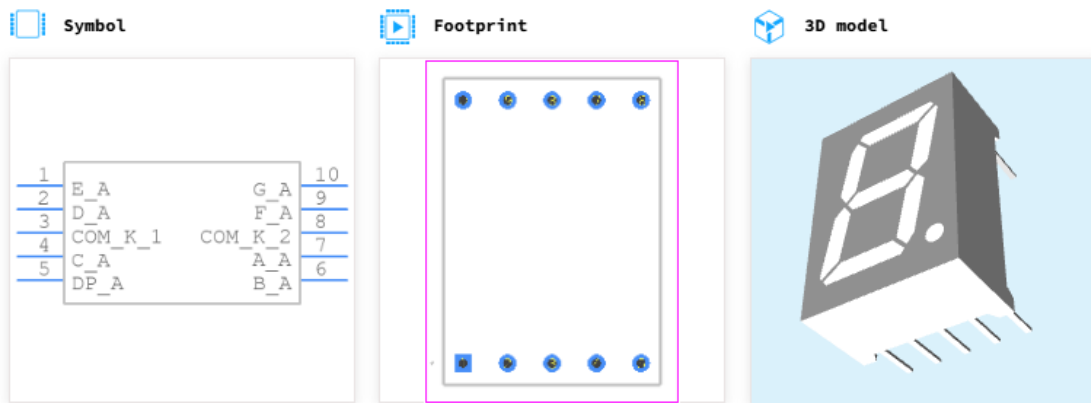


Figura 2 - Pesquisa dos componentes

#### 7.1.2. 2ºPasso: Importação das bibliotecas

De seguida, as bibliotecas descarregadas foram extraídas e importadas para o Altium Designer. Após a validação dos *footprints* e dos símbolos esquemáticos, foi possível prosseguir com o desenvolvimento dos esquemáticos do circuito.

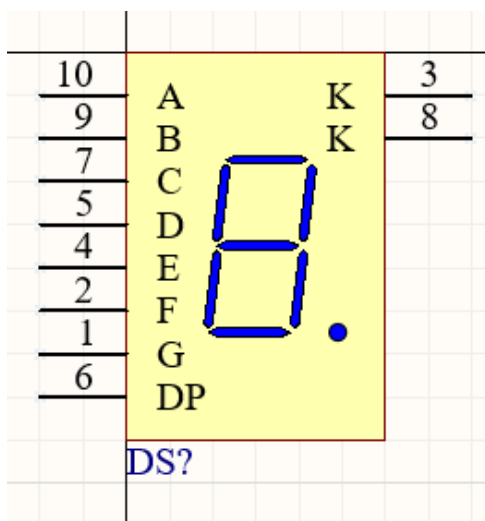


Figura 4 - Schematic Display

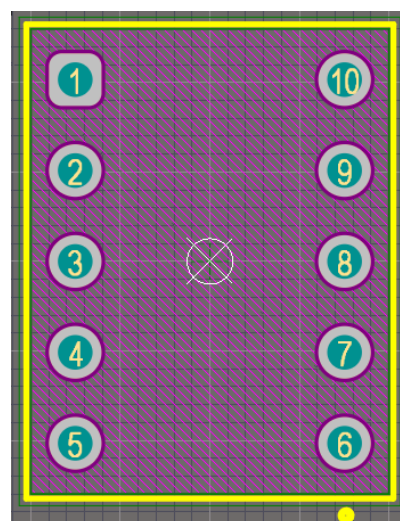


Figura 3 - Footprint do display

### 7.1.3. 3ºPasso – Ajusto dos PADS

Em alguns componentes, foi necessário proceder ao ajuste manual dos *pads*, de forma a garantir dimensões adequadas para a soldadura dos componentes. Estes ajustes permitiram evitar problemas durante a soldadura da PCB.

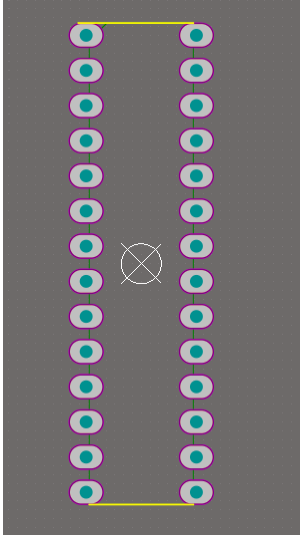


Figura 5 - Pads do ATmega328P

## 7.2. Organização da PCB

O layout da PCB foi desenvolvido no Altium Designer, recorrendo à funcionalidade *Rooms* para organizar o projeto por blocos funcionais. As ligações foram inicialmente realizadas dentro de cada *room*, correspondendo a um bloco do sistema, e posteriormente foi efetuado o encaminhamento global da PCB, interligando todos os componentes.

Esta metodologia permitiu uma melhor organização do layout, facilitou o encaminhamento das pistas e contribuiu para a redução de erros no desenvolvimento do hardware.

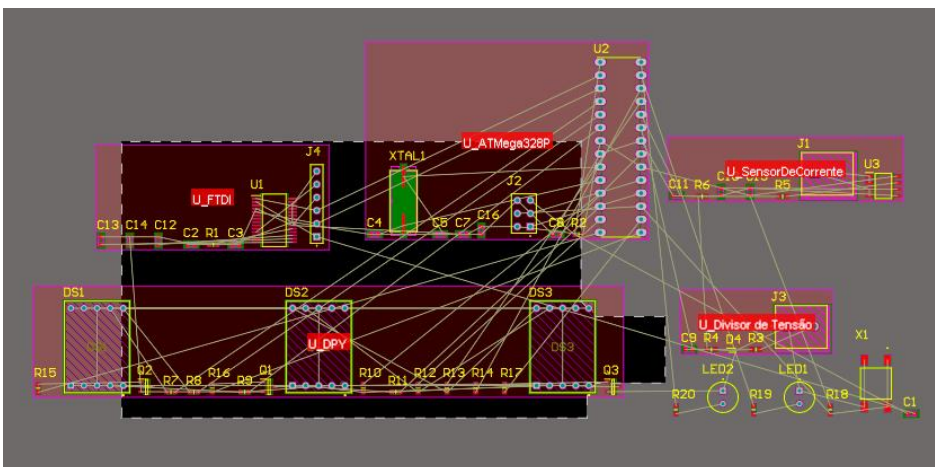


Figura 6 - Rooms do Altium

### 7.3. PCB

Depois de finalizarmos os esquemáticos e o layout da PCB, exportámos os ficheiros de fabrico (Gerber files e NC Drills) a PCB foi para produção. Após recebermos a PCB pronta, procedemos à soldadura de todos os componentes, incluindo o microcontrolador, sensor de corrente, FT232RL, divisores de tensão, displays e restantes elementos, garantindo que todas as ligações estivessem corretas e que o circuito estivesse pronto para testes e funcionamento.

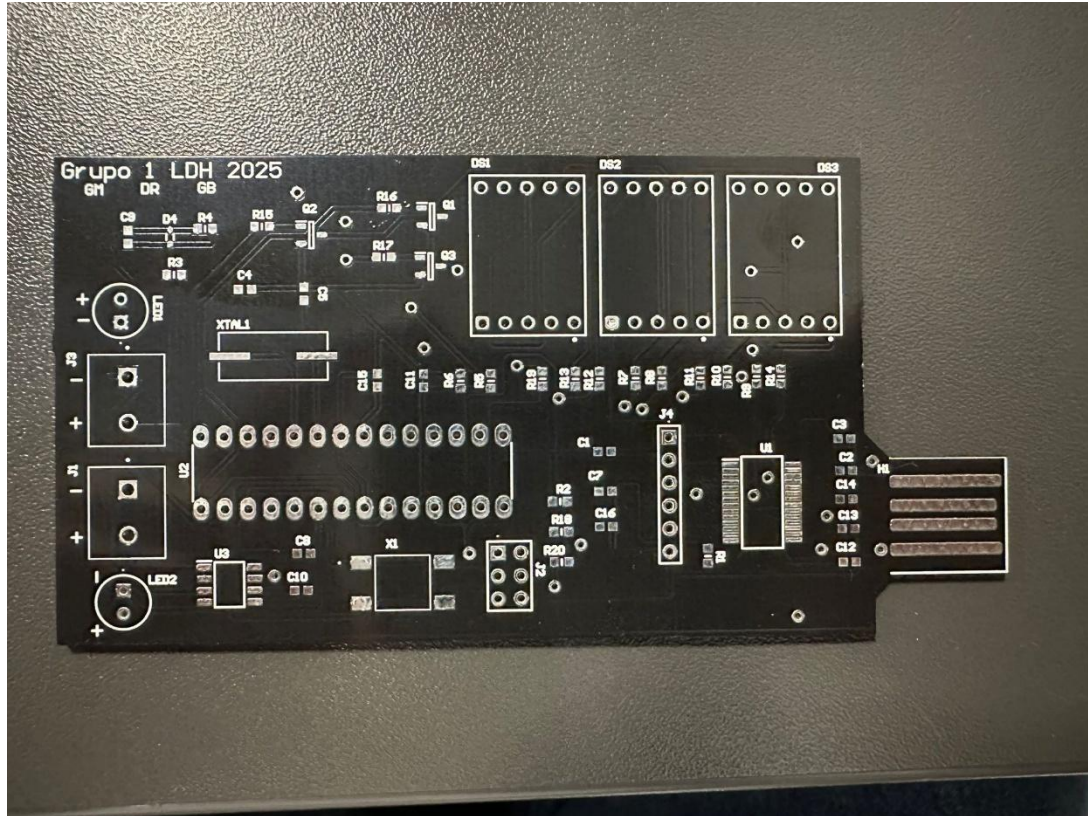


Figura 7 - PCB sem componentes

## 7.4. ATmega328P

O microcontrolador ATmega328P constitui o núcleo do sistema, sendo responsável pela coordenação de todas as funcionalidades do multímetro. Este dispositivo integra um conversor analógico-digital (ADC) de 10 bits, temporizadores internos, múltiplas portas de entrada e saída digitais e uma interface de comunicação série UART, características que o tornam adequado para aplicações de instrumentação eletrónica.

No presente projeto, o ATmega328P desempenha as seguintes funções principais:

- Aquisição dos sinais analógicos provenientes do circuito de medição de tensão e do sensor de corrente, através do ADC interno;
- Processamento das leituras e cálculo das médias configuráveis, com o objetivo de reduzir o ruído e aumentar a estabilidade das medições;
- Controlo da multiplexação dos três displays de 7 segmentos e da indicação do modo de funcionamento através de LEDs;
- Comunicação com um computador através da interface UART, permitindo o envio das medições e a receção de comandos de configuração.

O microcontrolador opera com um cristal externo de 16 MHz, garantindo maior precisão temporal e fiabilidade na comunicação série.



Figura 8 - ATmega328P

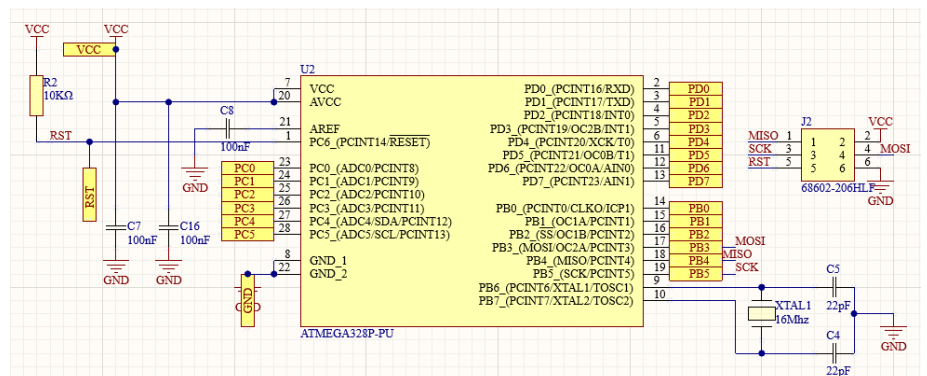


Figura 9 - Schematic ATmega328P

## 7.5. FTDI

A comunicação entre o microcontrolador e o computador é realizada através do circuito integrado FT232RL, responsável pela conversão entre os sinais USB e UART. Este dispositivo permite que o sistema seja reconhecido pelo computador como uma porta COM, possibilitando a troca de dados com aplicações de monitorização.

Funções principais:

### 1. Conversão USB ↔ UART

- O microcontrolador fala em UART (TX, RX), mas os computadores modernos usam USB.
- O FT232RL traduz esses sinais, permitindo que o PC “veja” o microcontrolador como uma porta COM.

### 2. Alimentação e regulação

- Pode fornecer 3,3V ou 5V ao circuito, dependendo da configuração, ou apenas atuar como interface sem alimentação.

### 3. Drivers FTDI

- Para o computador reconhecer o chip, é necessário instalar os drivers da FTDI, que criam uma porta COM.

O conector J4 de 6 pinos serve para ligar o circuito a outro dispositivo via UART. Ele fornece alimentação (VCC/GND), permite a comunicação serial (TXD/RXD) e inclui o DTR, usado para controlar o reset de microcontroladores durante programação.

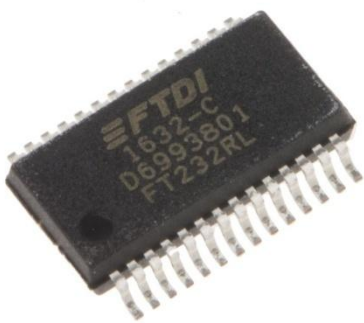


Figura 10 - FT232RL

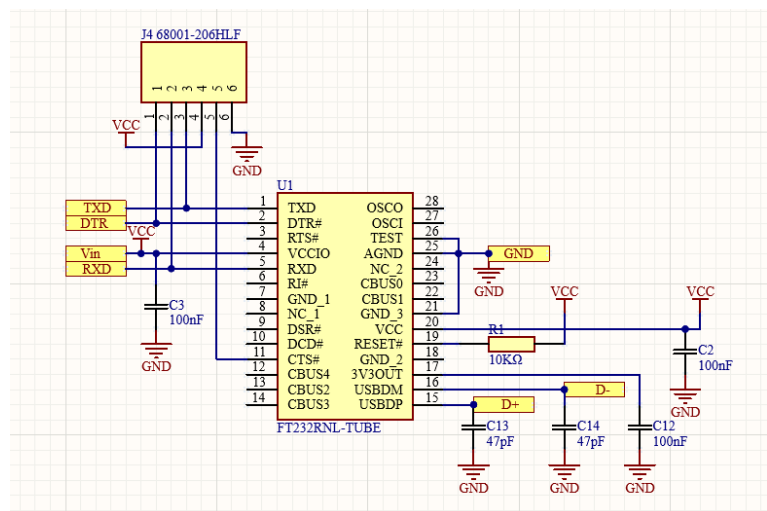


Figura 11 - Schematic do FTDI

## 7.6. Sensor de corrente

A medição de corrente é realizada utilizando o sensor de efeito Hall ACS712, na versão de 5 A. Este sensor fornece à saída uma tensão analógica proporcional à corrente que atravessa o seu condutor interno.

Para corrente nula, a tensão de saída é aproximadamente 2,5 V. A sensibilidade do sensor é de 0,185 V/A, o que significa que a tensão de saída varia cerca de 185 mV por ampere. A saída do sensor é ligada diretamente a uma das entradas do ADC do ATmega328P.

A corrente é calculada por software segundo a expressão:

$$I = (V\_ADC - 2,5) / 0,185$$

Considerando a resolução de 10 bits do ADC e a gama de funcionamento pretendida, é possível obter medições de corrente até 3 A com uma resolução aproximada de 0,1 A, cumprindo os requisitos definidos.



Figura 12 - ACS712

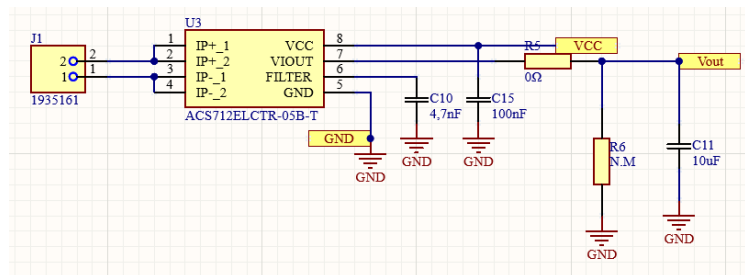


Figura 13 - Schematic Sensor de corrente

## 7.7. Divisor de tensão

O conversor analógico-digital do ATmega328P suporta tensões de entrada apenas até ao valor da sua tensão de referência, que neste projeto é de 5V. No entanto, o sistema deverá medir tensões externas até 30V, tornando necessário o condicionamento do sinal antes da sua aplicação ao microcontrolador.

Para este efeito, foi implementado um divisor de tensão, constituído por duas resistências de valores  $R1 = 100\text{ k}\Omega$  e  $R2 = 20\text{ k}\Omega$ . Este divisor reduz a tensão de entrada para um valor proporcional e seguro para o ADC, garantindo que, mesmo para a tensão máxima de 30 V, a tensão aplicada à entrada analógica não exceda os 5 V.

A tensão de saída é dada por:

$$V_{out} = V_{in} \times \frac{R2}{R1 + R2}$$

Substituindo:

$$V_{out} = 30 \times \frac{20k}{120k} = 5V$$

Numa fase inicial do desenvolvimento, foi incluído um diodo Zener no circuito de medição de tensão, com o objetivo de proteger a entrada do ADC contra sobretensões. No entanto, durante a fase de testes, verificou-se que a presença do diodo introduzia erros nas medições. Por este motivo, optou-se por remover o diodo Zener do circuito.

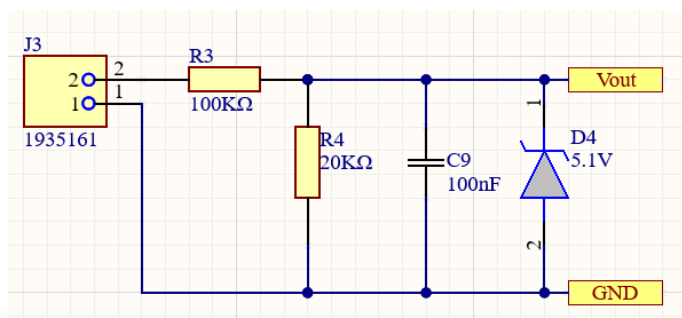


Figura 14 - Schematic divisor de tensão

## 7.8. Display's

Foram utilizados três displays de 7 segmentos do tipo cátodo comum. Na multiplexação, os segmentos A–G são partilhados entre os displays, sendo ativado apenas um display de cada vez através do seu cátodo comum.

A comutação rápida entre displays cria a percepção de todos estarem ligados simultaneamente, reduzindo o número de pinos necessários no microcontrolador.

Cada display é ligado ao GND através de um transístor NPN, permitindo conduzir a corrente total dos segmentos sem sobrecarregar os pinos do microcontrolador.

Considerando uma corrente máxima de aproximadamente 70 mA por display, foi escolhido um transístor adequado.

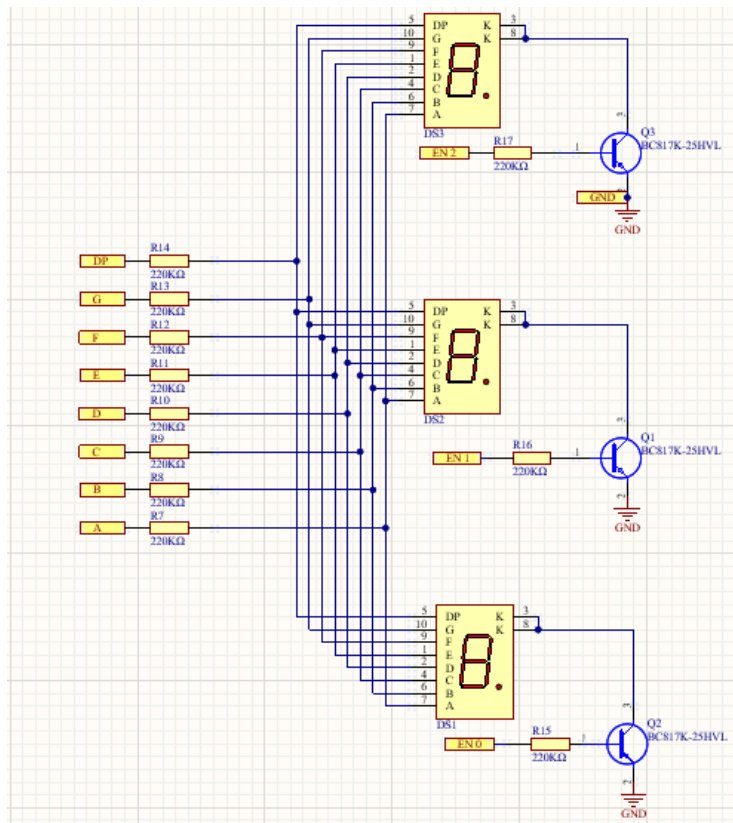


Figura 15 - Schematic dos display's

## 7.9. LED's e Botões

- LED de Tensão (LED1)

Este LED é utilizado para saber que estamos a medir tensão. Está ligado entre a linha de sinal e o GND, em série com um resistor limitador de corrente de  $220\ \Omega$ , garantindo que a corrente que atravessa o LED se mantém dentro dos valores seguros de funcionamento.

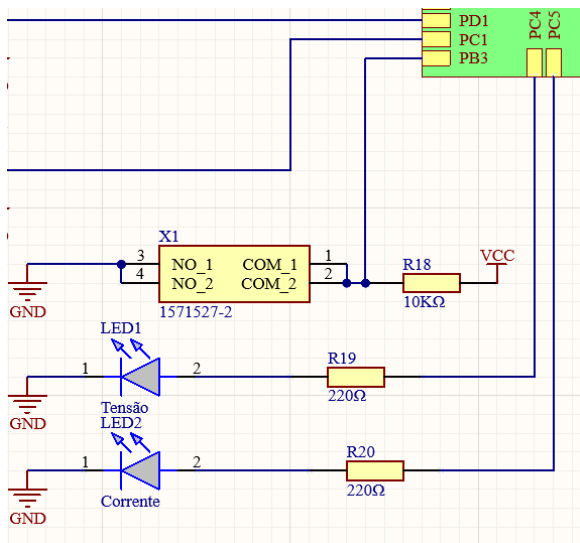
- LED de Corrente (LED2)

O segundo LED é utilizado para saber que estamos a medir corrente. Tal como o LED de tensão, encontra-se ligado ao GND através de um resistor de  $220\ \Omega$ , funcionando também como proteção contra sobrecorrente.

O circuito inclui um botão utilizado para controlo do sistema (por exemplo, para seleccionar um modo de funcionamento):

- Um dos lados do interruptor está ligado ao GND.
- O outro lado está ligado à entrada do microcontrolador, sendo mantido num nível lógico definido através de um resistor pull-up de  $10\ k\Omega$  (R18) conectado ao VCC.

Esta configuração garante que:



- Quando o botão não está acionado, a entrada do microcontrolador permanece em nível lógico alto.
- Quando o botão é pressionado, a entrada é ligada ao GND, passando para nível lógico baixo.

Figura 16 - LED's e botões

## 7.10. Esquema final

O esquema final apresenta a integração completa do sistema, tendo como elemento central o microcontrolador ATmega328P, responsável pela leitura dos sensores e pelo controlo dos periféricos.

A alimentação é fornecida através de uma ligação USB, sendo também utilizada para a comunicação série com o microcontrolador por intermédio do módulo FTDI, que permite a programação do sistema.

O circuito inclui um divisor de tensão e um sensor de corrente, ambos ligados a entradas analógicas do ATmega328P.

Para garantir a estabilidade do funcionamento, é utilizado um condensador de 100 nF no circuito de reset. O sistema dispõe ainda de um interruptor.

Foram integrados dois LEDs indicadores, um para sinalização da presença de tensão e outro para indicação de corrente.

Por fim, os display's (DPY) encontram-se ligado aos pinos digitais do microcontrolador, permitindo a visualização de informação relevante ao utilizador.

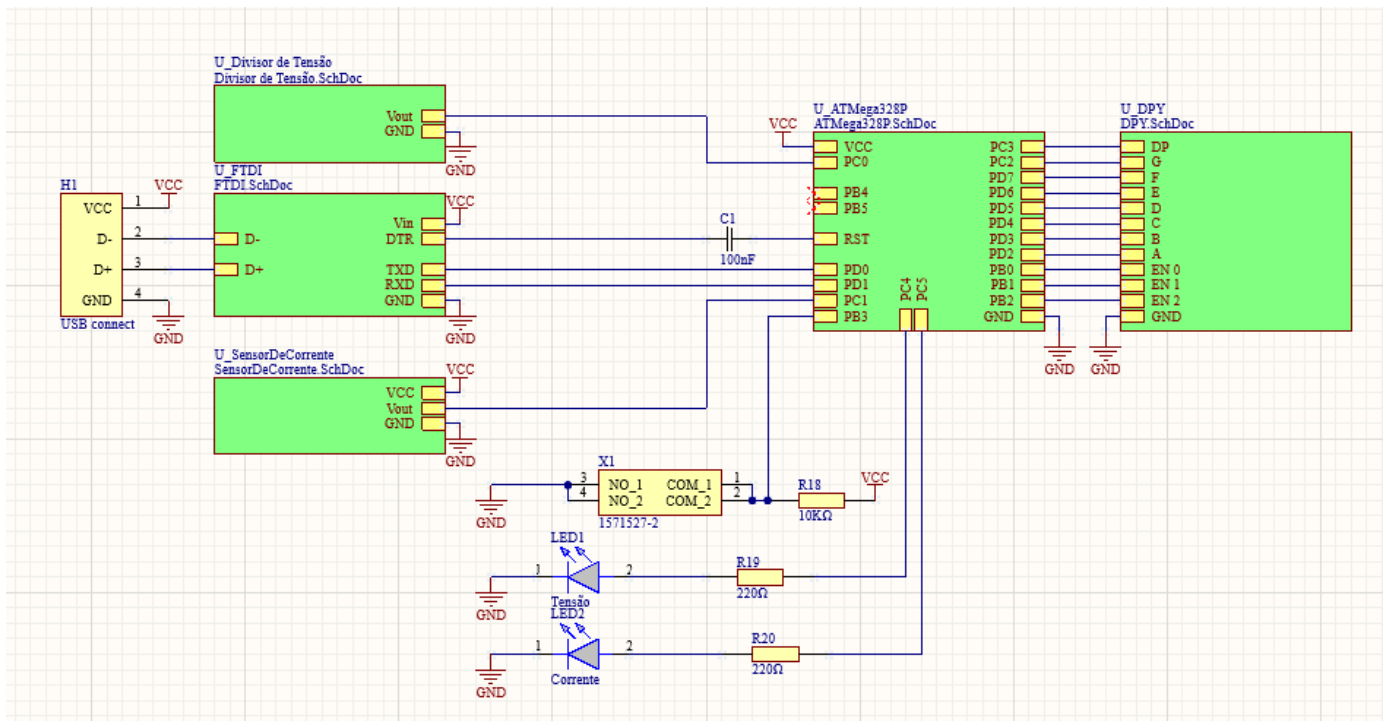


Figura 17 - Esquema final

## 8. Software

O firmware foi desenvolvido em linguagem C, utilizando o Visual Studio Code. O software é responsável pela leitura do ADC, cálculo das médias, atualização dos displays, processamento de comandos recebidos pela porta série e envio dos dados para o computador.

A multiplexação dos displays é realizada através de interrupções periódicas geradas por um temporizador, garantindo uma atualização estável sem bloquear a execução do programa principal.

### 8.1. Definição das bibliotecas

```
1  #define F_CPU 16000000UL           // Define a frequência do relógio do ATmega328P (16MHz)
2  #include <avr/io.h>                // Biblioteca padrão para entrada/saída de pinos
3  #include <avr/interrupt.h>         // Biblioteca para manipulação de interrupções
4  #include <util/delay.h>            // Biblioteca para funções de atraso (delay)
5  #include <math.h>                  // Biblioteca para funções matemáticas (fabsf)
6  #include <stdio.h>                 // Biblioteca para entrada/saída formatada (sprintf)
7  #include <stdlib.h>                // Biblioteca padrão para conversões (atoi)
8
9  #define BAUDRATE 9600              // Define a velocidade da comunicação serial
10 #define MYUBRR ((F_CPU / (16UL * BAUDRATE)) - 1) // Cálculo do registo para o Baudrate
```

Figura 18 - Definição das bibliotecas

- Diz ao microcontrolador a velocidade do relógio (16 MHz);
- Inclui bibliotecas que permitem usar pinos, interrupções, atrasos, cálculos e enviar texto para o computador;

### 8.2. Variáveis globais

```
13 volatile uint8_t d[3] = {0, 0, 0}; // Armazena os 3 dígitos a serem exibidos
14 volatile uint8_t digito_atual = 0;  // Controla qual dígito está a ser refrescado agora
15 volatile uint16_t amostras_V = 500; // Número de amostras para média de Tensão
16 volatile uint16_t amostras_A = 500; // Número de amostras para média de Corrente
```

Figura 19 - Variáveis globais

- d[3] → guarda os números que vão aparecer nos 3 displays;
- digito\_atual → indica qual dígito está aceso agora;
- amostras\_V e amostras\_A → dizem quantas leituras serão feitas para a média;

### 8.3. Padrão de bits dos segmentos dos display's

```
19 uint8_t digitos_map[10] = {
20     0b00111111, 0b00000110, 0b01011011, 0b01001111, 0b01100110,
21     0b01101101, 0b01111101, 0b00000111, 0b01111111, 0b01101111
```

Figura 20 - Padrão de bits dos segmentos

- Cada número de 0 a 9 tem um padrão de bits para acender os segmentos do display;
- Por exemplo, 0b00111111 acende os segmentos A–F para formar o número “0”;

## 8.4. Atualização dos display's (interrupção do timer)

```

25 ISR(TIMER1_COMPA_vect) {
26     // Apaga os pínos PB0, PB1 e PB2 (desliga os 3 dígitos para evitar brilho fantasma)
27     PORTB &= ~((1<<PB0)|(1<<PB1)|(1<<PB2));
28
29     // Obtém o padrão de bits do dígito atual no array
30     uint8_t valor = digitos_map[d[digito_atual]];
31
32     // Atualiza PORTD (segmentos A a F nos pínos PD2 a PD7), mantendo PD0 e PD1 (UART)
33     PORTD = (PORTD & 0x03) | ((valor & 0x3F) << 2);
34
35     // Se o bit 6 (Segmento G) estiver ativo, liga PC2, senão desliga
36     if ((valor >> 6) & 1) PORTC |= (1 << PC2); else PORTC &= ~(1 << PC2);
37
38     // Liga o Ponto Decimal (PC3) apenas quando estamos no segundo dígito (PB1)
39     if (digito_atual == 1) PORTC |= (1 << PC3); else PORTC &= ~(1 << PC3);
40
41     // Lógica para ligar o dígito físico e supressão de zero à esquerda
42     if (digito_atual == 0) {
43         if (d[0] > 0) PORTB |= (1 << PB0); // Só liga o 1º dígito se o valor for >= 10.0
44     }
45     else if (digito_atual == 1) {
46         PORTB |= (1 << PB1); // Segundo dígito sempre liga (ex: 0.X)
47     }
48     else if (digito_atual == 2) {
49         PORTB |= (1 << PB2); // Terceiro dígito sempre liga
50     }
51
52     // Alterna para o próximo dígito (0 -> 1 -> 2 -> 0...)
53     digito_atual = (digito_atual + 1) % 3;
54 }

```

Figura 21 - Atualização dos display's

- Este pedaço é como um despertador que toca a cada 2 ms;
- Ele acende só um dígito de cada vez e coloca os segmentos corretos;
- Depois passa para o próximo dígito;
- Faz isso rápido, assim o olho vê todos os dígitos ligados ao mesmo tempo;

## 8.5. Inicialização serial

```

57 void uart_init(unsigned int ubrr) {
58     UBRR0H = (unsigned char)(ubrr >> 8); // Define a parte alta do baudrate
59     UBRR0L = (unsigned char)ubrr; // Define a parte baixa do baudrate
60     UCSR0B = (1 << RXEN0) | (1 << TXEN0); // Ativa o receptor e o transmissor serial
61     UCSR0C = (3 << UCSZ00); // Define formato: 8 bits de dados, 1 stop bit
62 }

```

Figura 22 - Inicialização serial

- Configura a comunicação com o computador (Serial);
- Permite enviar e receber texto;

## 8.6. Função para enviar texto para o PC

```
65 void uart_print(char* s) {  
66     while (*s) { // Enquanto houver caracteres na string  
67         while (!(UCSR0A & (1 << UDRE0))); // Espera o buffer de transmissão ficar vazio  
68         UDR0 = *s++; // Coloca o caractere no buffer para enviar  
69     }  
70 }
```

Figura 23 - Função para enviar texto para o PC

- É como falar com o computador.
- Recebe uma mensagem (string) e envia letra por letra.

## 8.7. Função para processar comandos recebidos

```
73 void processa_entrada_serial() {  
74     if (UCSR0A & (1 << RXC0)) { // Se houver dados novos para ler  
75         char tipo = UDR0; // Lê o primeiro caractere (V ou A)  
76         _delay_ms(30); // Espera o resto dos dígitos chegarem  
77         char buf[10]; uint8_t i = 0; // Buffer temporário para o número  
78         while ((UCSR0A & (1 << RXC0)) && i < 9) buf[i++] = UDR0; // Lê o valor numérico  
79         buf[i] = '\0'; // Finaliza a string do buffer  
80         uint16_t val = atoi(buf); // Converte o texto recebido em número inteiro  
81         if (val > 0) {  
82             if (tipo == 'V' || tipo == 'v') {  
83                 amostras_V = val; // Atualiza o limite de média da Tensão  
84                 uart_print("\r\n[OK] Media Tensao atualizada.\r\n");  
85             } else if (tipo == 'A' || tipo == 'a') {  
86                 amostras_A = val; // Atualiza o limite de média da Corrente  
87                 uart_print("\r\n[OK] Media Corrente atualizada.\r\n");  
88             }  
89         }  
90     }  
91 }
```

Figura 24 - Função para processar comandos recebidos

- Imagina que o computador envia um recado como "V500" ou "A200";
- V ou A indica se é tensão ou corrente;
- Os números dizem quantas leituras usar para a média;
- O microcontrolador lê o comando, guarda o número na variável certa e envia uma mensagem de confirmação para o computador;
- Exemplo: "V500" → agora o programa vai calcular a média da tensão usando 500 leituras;

## 8.8. Função leitura do ADC

```

94  uint16_t adc_leitura(uint8_t canal) {
95      uint32_t soma = 0; // Variável para somar amostras
96      ADMUX = (1 << REFS0) | (canal & 0x0F); // Define referência 5V e o canal do pino
97      _delay_us(50); // Tempo para estabilizar a tensão no pino
98      for(uint8_t i=0; i<64; i++) { // Faz 64 leituras rápidas para reduzir ruído
99          ADCSRA |= (1 << ADSC); // Inicia a conversão
100         while (ADCSRA & (1 << ADSC)); // Espera terminar
101         soma += ADC; // Acumula o valor lido
102     }
103     return (uint16_t)(soma / 64); // Retorna a média das 64 leituras
104 }

```

Figura 25 - Leitura do ADC

- Lê a tensão ou corrente várias vezes para evitar erros;
- Calcula a média dessas leituras para ter um valor mais estável;
- canal diz qual pino do ADC que vai ler (0 = tensão, 1 = corrente);

## 8.9. Função principal

```

107 int main(void) {
108     // Configuração de Direção de Pinos (1=Saída, 0=Entrada)
109     DDRD |= 0b11111100; // Pinos PD2 a PD7 como saída (segmentos)
110     DDRC |= 0b00111100; // Pinos PC2 a PC5 como saída (seg G, ponto e LEDs)
111     DDRB |= 0b0000111; // Pinos PB0 a PB2 como saída (controlo dígitos)
112     PORTB |= (1 << PB3); // Ativa Pull-up interno no pino do Botão (PB3)
113
114     // Configuração do Timer 1 para refrescar o Display
115     TCCR1B |= (1 << WGM12) | (1 << CS11); // Modo CTC, Prescaler de 8
116     OCR1A = 3999; // Define teto para contar até 4000 (2ms)
117     TIMSK1 |= (1 << OCIE1A); // Ativa interrupção por comparação
118
119     uart_init(MYUBRR); // Inicializa a porta Serial
120     ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0); // Ativa ADC com prescaler
121     sei(); // Ativa as interrupções globais
122
123     uint8_t modo = 0; // 0 = Tensão, 1 = Corrente
124     uint8_t btn_last = 1; // Estado anterior do botão (pull-up = 1 solto)
125     char buffer[128]; // Buffer para criar as mensagens de texto
126     float acum_serial = 0; // Acumulador para a média do Serial
127     uint16_t count_serial = 0; // Contador de amostras feitas para o Serial

```

Figura 26 - Função principal

### 1 Configuração dos pinos:

- DDRD (PD2–PD7) → controla segmentos A–F do display.
- DDRC (PC2–PC5) → controla segmento G, ponto decimal e LEDs de tensão/corrente.
- DDRB (PB0–PB2) → seleciona qual dígito do display está ligado (multiplexação).
- PB3 → botão com Pull-up interno.

## 2 Timer 1:

- Contador que dispara uma interrupção a cada 2ms (OCR1A = 3999) para atualizar o display sem travar o programa.
- Modo CTC e prescaler de 8.

## 3 Serial (UART):

- `uart_init(MYUBRR)` → inicializa comunicação com PC a 9600 bps.

## 4 ADC:

- Liga o conversor analógico (ADEN = 1) e define prescaler 128 para ler sensores de tensão e corrente.

## 5 Interrupções globais:

- `sei()` → permite que o Timer atualize o display automaticamente.

## 6 Variáveis do loop:

- `modo` → seleciona tensão (0) ou corrente (1).
- `btn_last` → guarda estado anterior do botão (para detectar clique).
- `buffer` → para mensagens Serial.
- `acum_serial` e `count_serial` → acumulam leituras para calcular média.

# 8.10. Loop

## 8.10.1. Leitura do botão

```

132 // Lógica de leitura do Botão (Debounce simples)
133 uint8_t btn_now = (PINB & (1 << PB3)) != 0;
134 if (btn_last && !btn_now) { // Se detectou clique (transição de 1 para 0)
135     modo ^= 1; // Inverte o modo (V <-> A)
136     acum_serial = 0; // Reseta a média ao trocar de modo
137     count_serial = 0; // Reseta o contador
138     _delay_ms(200); // Pausa para evitar múltiplos cliques
139 }
140 btn_last = btn_now; // Salva o estado atual para o próximo ciclo
141
142 float valor_inst = 0; // Armazena a leitura instantânea calculada
143 uint16_t limite = (modo == 0) ? amostras_V : amostras_A; // Escolhe limite conforme
144
145 if (modo == 0) { // --- MODO TENSÃO ---
146     PORTC |= (1 << PC4); // Liga LED de Voltagem
147     PORTC &= ~(1 << PC5); // Desliga LED de Corrente
148     valor_inst = adc_leitura(0) * 0.0299; // Converte bits para Tensão (ajusta se n
149 } else { // --- MODO CORRENTE ---
150     PORTC |= (1 << PC5); // Liga LED de Corrente
151     PORTC &= ~(1 << PC4); // Desliga LED de Voltagem
152     uint16_t r = adc_leitura(1); // Lê o canal ADC1
153     valor_inst = fabsf(((r * 5.0 / 1024.0) - 2.5) / 0.185); // Cálculo para sensor
154     if (valor_inst < 0.05) valor_inst = 0; // Filtro para ignorar pequenos ruídos
155 }

```

Figura 27 - Leitura do botão

## 1. Controlo pelo Botão

- Detecta pressão do botão (PB3);
- Alterna modo entre 0 (Tensão) e 1 (Corrente);
- Reseta o acumulador da média para evitar mistura de valores;

## 2. Configuração da Média

- Define limite de amostras (amostras\_V ou amostras\_A) para cada modo, configurável;

## 3. Cálculo de Tensão (modo 0)

- Liga LED de Voltagem, desliga LED de Corrente;
- Lê ADC0 e multiplica por 0,0299 para obter Volts;

## 4. Cálculo de Corrente (modo 1)

- Liga LED de Corrente, desliga LED de Voltagem;
- ACS712: converte ADC1 em Volts, subtrai 2,5 V e divide por 0,185 V/A;
- Valores < 0,05 A são zerados para reduzir ruído;

### 8.10.2. Atualização dos dados nos display's

```
158 uint16_t n = (uint16_t)(valor_inst * 10 + 0.5); // Arredonda para 1 casa decimal
159 if (n > 999) n = 999; // Limita ao máximo do display (99.9)
160 d[0] = (n / 100) % 10; // Dezena
161 d[1] = (n / 10) % 10; // Unidade
162 d[2] = n % 10; // Decimal
```

Figura 28 - Atualização de dados nos display's

## 1. Processamento dos Números

- Arredonda para 1 casa decimal (multiplica por 10 e arredonda);
- Limita valor máximo a 99,9 para o display;
- Separa o número em Dezena, Unidade e Decimal (variáveis d[0], d[1], d[2]);

## 2. Lógica e Medição

- Botão alterna entre Volts e Amperes, limpando médias para evitar erros;
- Configura filtro de amostras via Serial para V ou A;
- Cálculo de tensão: leitura do sensor × 0,0299;
- Cálculo de corrente: ACS712 → converte bits para Volts, ajusta desvio de 2,5 V e divide pela sensibilidade;

- Filtro: ignora ruídos < 0,05 A para estabilidade no display;

### 8.10.3. Cálculo da média para o serial monitor

```
164 // --- CÁLCULO DA MÉDIA PARA O SERIAL ---
165 acum_serial += valor_inst; // Soma o valor atual ao acumulador
166 count_serial++; // Incrementa contador de amostras
167
168 if (count_serial >= limite) { // Quando atingir o número de amostras definido
169     float v_med = acum_serial / count_serial; // Calcula a média real
170
171     // Divide floats em Parte Inteira e Decimal para o sprintf (ATmega gasta menos m
172     int i_inst = (int)valor_inst; int d_inst = (int)((valor_inst - i_inst) * 100);
173     int i_med = (int)v_med; int d_med = (int)((v_med - i_med) * 100);
174
175     // Monta a mensagem final: [Modo] Valor Instantâneo | Valor Médio
176     sprintf(buffer, "[%s] Agora: %d.%02d | Media(%u): %d.%02d\r\n",
177         (modo == 0 ? "V" : "A"), i_inst, d_inst, limite, i_med, d_med);
178
179     uart_print(buffer); // Envia a mensagem para o Monitor Serial
180     acum_serial = 0; // Limpa acumulador para nova média
181     count_serial = 0; // Limpa contador para nova média
182 }
183 _delay_ms(1); // Pequena pausa para estabilidade do loop princ
184 }
185 }
```

Figura 29 - Cálculo da média para o serial monitor

- Seleção de Medição: Botão alterna entre tensão (V) e corrente (A), limpando médias para novos cálculos;
- Cálculo de Tensão (V): Leitura do sensor × 0,0299;
- Cálculo de Corrente (A): Fórmula do ACS712; ignora ruídos < 0,05 A;
- Formatação para Display: Arredonda para 1 casa decimal e separa em dezena, unidade e decimal;
- Média e Saída Serial: Acumula leituras, calcula média real e envia ao PC mostrando valor instantâneo vs. Médio;
- Envio: Calcula a média real e envia para o PC (Monitor Serial) mostrando o valor Instantâneo vs Médio.

## 9. Testes

Foram realizados testes funcionais para validar o correto funcionamento do sistema, incluindo a medição de diferentes valores de tensão e corrente e a verificação da estabilidade das leituras. Os resultados obtidos demonstram um comportamento consistente, embora se tenham observado limitações na comunicação série ao baud rate pretendido.

### 9.1. Corrente

Após a conclusão da fabricação da PCB e a soldadura de todos os componentes, foram realizados testes funcionais com o objetivo de verificar o correto funcionamento do sistema. Um dos testes efetuados foi o teste de medição de corrente.

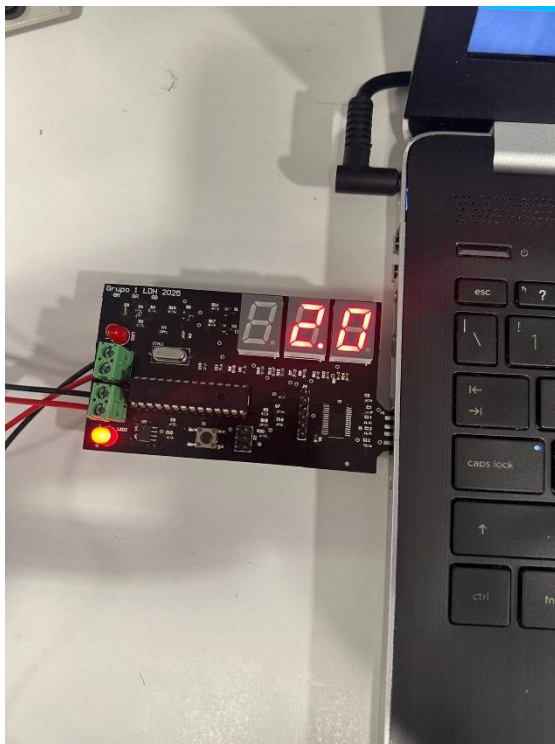


Figura 31 - Teste funcional de corrente

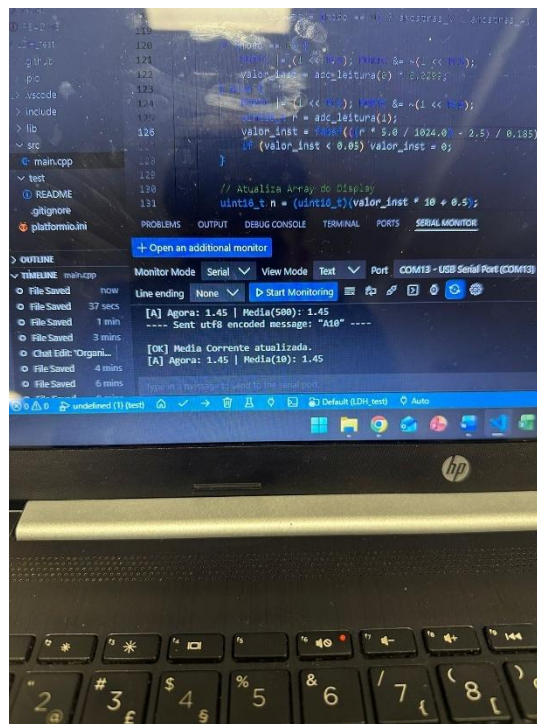


Figura 30 - Teste da media Monitor Serial

## 9.2. Tensão

Após a PCB estar concluída foram realizados testes funcionais ao circuito de medição de tensão. Inicialmente, foi utilizada uma pilha como fonte de alimentação, permitindo verificar o correto funcionamento do sistema para baixos valores de tensão.

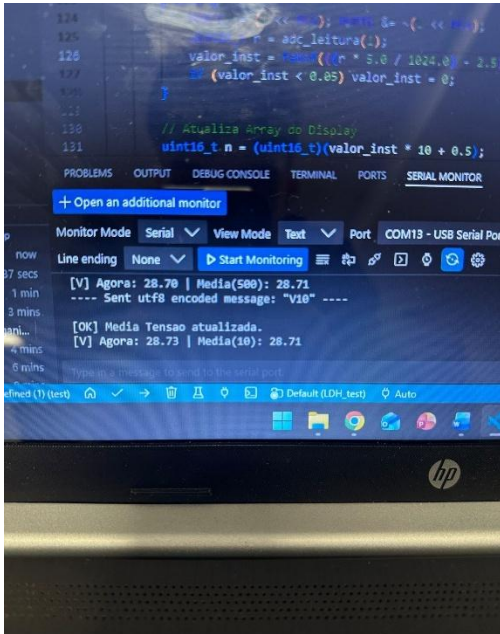


Figura 34 - Teste do comando no monitor serial

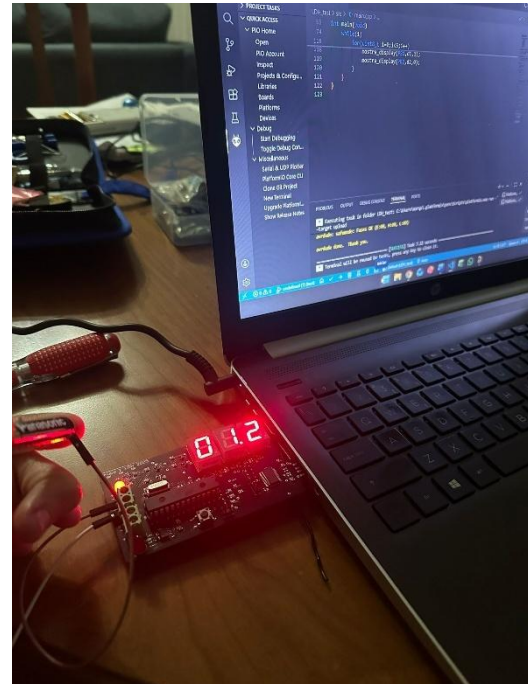


Figura 33 - Teste com a pilha

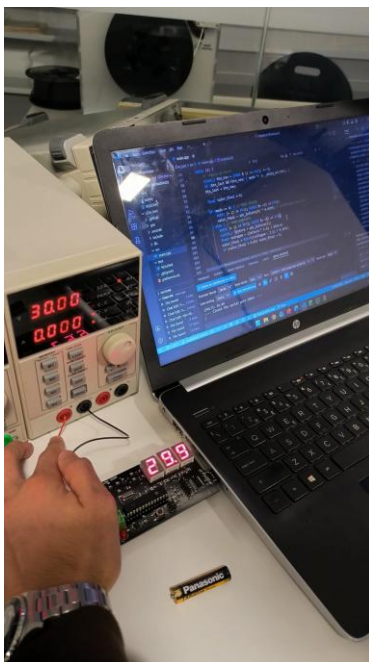
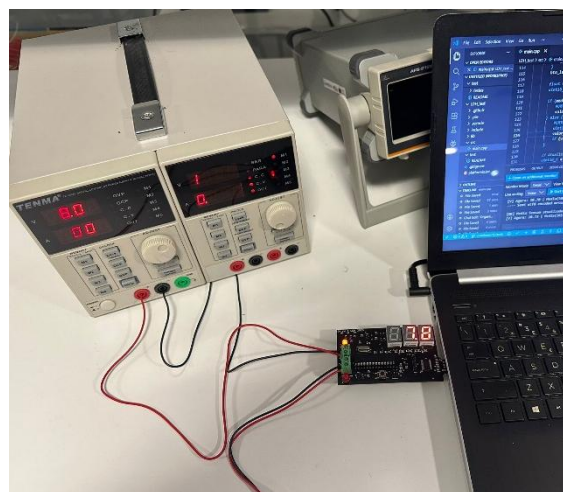


Figura 32 - Teste na fonte de alimentação



## 10. PCB Final

Nas figuras seguintes é apresentada a PCB final do multímetro com datalogger, após a conclusão do fabrico, montagem e soldadura de todos os componentes, encontrando-se o sistema pronto para funcionamento e testes.

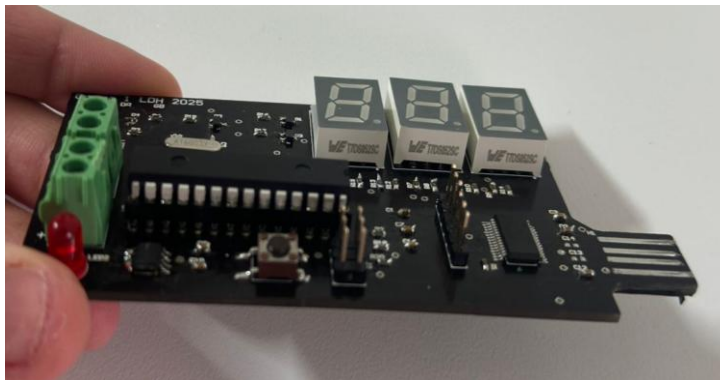
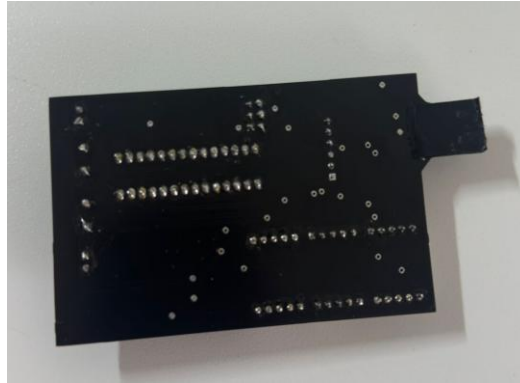
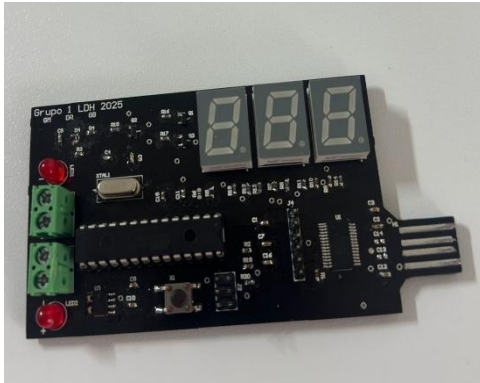


Figura 35 - PCB Final

## **11. Conclusão**

Com a realização deste trabalho foi possível aplicar, de forma prática, os conhecimentos adquiridos na área de desenvolvimento de hardware e sistemas eletrónicos embebidos. O projeto permitiu consolidar competências em análise, design, implementação e validação de sistemas eletrónicos, bem como em trabalho de equipa e organização de projeto. Durante o desenvolvimento, enfrentámos algumas dificuldades, nomeadamente na configuração do software, na definição correta do baud rate e na implementação das médias para o Serial Monitor, que exigiram ajustes e testes para garantir leituras fiéis e estáveis.